A coded set model of PTS-rk

October 20, 2026

Abstract

The reference set model interprets every derivation of the original, annotated rank-based pure type system. Countable universes contain codes for finite sets, finite powerset iterates of a countable set, and the universes themselves. Dependent products are represented by cardinal codes, with a single coherent choice of coding bijections. The central proof difficulty is conversion: the interpretation is type-directed, so one must show that different original typing witnesses yield the same actual semantic value. A rank-controlled hereditary-substitution argument normalizes denotation certificates while preserving their values; confluence then supplies semantic conversion. This closes the original typing-soundness proof and gives an empty interpretation of Bottom. All hierarchy, classical, cardinal and choice parameters are explicit. Deriving the construction inside material BZC remains a separate development.

1 Scope and reading guide

This document describes the completed reference library `PTSrkModel`, whose sources are in `theories/ZFC_model`. That directory is preserved byte-for-byte at revision `ffa7131`; the exposition is consequently stored outside it. The manuscript reference is the section labelled `rk-model` in `latex/PTS-rk.tex` and `latex/paper.tex`. Its concise construction is expanded here into the infrastructure and proof obligations actually discharged by the Rocq development.

The final entry point is `ModelSoundness.v`. Its theorem `original_typing_sound` proves soundness of the original `Typing.typ`, and its `sound_nontrivial_model_exists` theorem constructs the semantic selector and combines soundness with emptiness of Bottom. Earlier module comments describing conversion or soundness as unfinished record intermediate stages; they are superseded by these public declarations.

The model depends on the original `PTSrk` kernel, selected normalization infrastructure, and the pinned `ZFC` library under `vendor/zfc`. Neither arithmetic interpretation is a dependency. The notation $A \simeq B$ below means existence of a set-graph bijection, and $A \preceq B$ means existence of a set-graph injection. Equality of semantic sets means the extensional relation `EQ` of the vendored library, not identity of Rocq representations.

2 Infrastructure and its division of work

The source tree is organized into several layers. Their order matters: the semantic conversion proof uses established syntactic metatheory and explicit set operations, and is subsequently supplied to typing soundness.

1. `SetFunctions`, `SetApplication`, `SetBijections` and `CodedFunctions` construct dependent function sets, graphs, application, bijections, inverse application and the coded beta law. Function sets are actual subsets of powersets, including the empty-domain and empty-fiber cases.
2. `CodeUniverses`, `CountableCodes`, `CardinalHierarchy`, `ProductEmbedding` and the cardinal-classification modules establish code decoding and the cardinal bound for dependent

products. `CountablePairing`, `CartesianMaps`, `PowersetSquares` and `CantorBernstein` supply the required injection and bijection calculus.

3. `HierarchyBounds`, `CodeHierarchyBounds` and `SuppliedHierarchy` bound the data inside an explicitly supplied envelope. `CodeWitnesses`, `SelectedCodes`, `GlobalCodeChoice` and `GlobalOperations` choose one coding witness uniformly and expose the three semantic operations.
4. `Denotation`, `InterpretedContexts` and the rule-realization modules define the original relational semantics and handle valuations, renaming, substitution, products, abstraction and application.
5. The `Canonical*` and `Ranked*` modules construct normal certificates, prove hereditary substitution and recover certificates from arbitrary original denotations. `ModelSoundness` packages the resulting theorems.
6. The `Bounded*` modules and `MaterialSetTheory`/`MaterialZermelo` record bounded constructions and an independent material deduction calculus. They are useful infrastructure, but their presence in this library does not constitute an internal proof of the whole model.

Auxiliary restricted certificates are proof tools. The final theorem is quantified over every original typing derivation and original valid valuation; users do not supply a canonical typing derivation or a normalization certificate as an additional premise.

3 Countable codes with large decodings

3.1 The universe construction

The three constructors of `CodeUniverses.Code` are `finite n`, `infinite j` and `sort j`. Their keys are $3n$, $3j+1$ and $3j+2$. Keys are represented by singleton numerals:

$$\nu_0 = \varnothing, \qquad \nu_{n+1} = \{\nu_n\}.$$

The theorem `repr_injective` proves that extensionally equal represented codes have identical constructors and indices. These numerals are not the usual von Neumann ordinals; their exact representation matters for the bounded constructions.

Let Ω be the set `omega_codes` of these numerals, and put $K_j = \mathcal{P}^j(\Omega)$. The actual definition of `carrier j` is this finite powerset iterate. Write c_n, d_j and s_j for the representations of finite, infinite and sort codes. Then

$$U_i = \{c_n : n \in \mathbb{N}\} \cup \{d_j : j \le i\} \cup \{s_j : j \in \mathbb{N}\}.$$

Decoding satisfies

$$\mathrm{El}(c_n) = \{\nu_k : k < n\}, \qquad \mathrm{El}(d_j) = K_j, \qquad \mathrm{El}(s_j) = U_j.$$

These equations are proved up to `EQ`. The decoder is defined on every set by a union over matching code witnesses, so it does not require a decision procedure for equality of arbitrary sets. `decoding_extensional` proves that it respects `EQ`; `decode_repr` proves the displayed laws.

Every U_i is countable, although its elements can decode to much larger sets. In particular $s_j \in U_0 = \mathrm{El}(s_0)$ for every j, which validates the original sort axiom $\mathcal{U}_j : \mathcal{U}_0$. This is a code

interpretation: it does not assert $U_0 \in U_0$. The theorem `decoding_bound` gives $\mathrm{El}(a) \preceq K_i$ whenever $a \in U_i$.

3.2 Products and the exact cardinal premises

For $a \in U_m$ and a family $b(x) \in U_n$, form the ordinary function set

$$F(a, b) = \prod_{x \in \mathrm{El}(a)} \mathrm{El}(b(x)).$$

The product-embedding infrastructure bounds this set by $K_{\max(m+1,n)}$. Cardinal classification then supplies a code c in $U_{\max(m+1,n)}$ and a bijection $F(a, b) \simeq \mathrm{El}(c)$. This is `decoded_product_code`; the increment of the domain rank is the same increment that occurs in the original PTS product rule.

The formal theorem retains three distinct ingredients. Classical excluded middle supports the case analysis. `cardinal_comparability` says that any two sets are comparable by injections. `infinite_powerset_gap` is exactly the implication

$$\Omega \preceq K, \quad K \preceq X, \quad X \preceq \mathcal{P}(K) \quad \Longrightarrow \quad X \preceq K \text{ or } \mathcal{P}(K) \preceq X.$$

The proof classifies an infinite set bounded by K_i by induction on i. Comparability places it below the preceding level or above it; the gap principle and Cantor–Bernstein handle the latter case. The finite case uses finite carriers. `family_choice` supplies the dichotomy that a set is finite or admits an injection of Ω, joining the cases. Neither initial-ordinal representatives nor a stronger unmentioned formulation of GCH are substituted for these actual premises.

4 A single choice of semantic operations

4.1 Why the choice must be independent of rank

A function set can have several cardinal codes and many coding bijections. Choosing a fresh bijection at every typing derivation would make the value of an abstraction depend on that derivation. `GlobalCodeChoice` instead indexes one selector by the actual extensional function set.

The construction assumes a transitive set W with $\Omega \subseteq W$ and a set H satisfying `tower_envelope W H`:

$$x \in H \quad \Longleftrightarrow \quad \exists n \in \mathbb{N},\ x \in \mathcal{P}^n(W).$$

`SuppliedHierarchy` derives transitivity and powerset closure of H, and bounds carriers, decoded sets and coding graphs. Eligible coding witnesses are pairs of codes and graphs, bounded by a fixed set built from all codes and $\mathcal{P}(H \times H)$. Ordinary set-indexed `family_choice` gives a selector on the relevant nonempty subsets. No class-choice operation or implicitly supplied hierarchy is used.

The selected code is admissible at every valid product bound: if its decoding is equinumerous with a set bounded by K_i, strict growth of the carriers excludes an infinite code with index larger than i. This is `bounded_code_admissible`. Thus a single selection works for all typing ranks. Extensional equality of input function sets yields equality of the selected witnesses by `global_choice_congruence`.

4.2 Product, abstraction and application

Let c_F and $f_F : F(a,b) \simeq \mathrm{El}(c_F)$ be the chosen code and bijection. The operations in `GlobalOperations` are

$$\begin{aligned} \mathrm{Prod}(a,b) &= c_F, \\ \mathrm{Lam}(a,b,g) &= f_F(\mathrm{graph}(g)), \\ \mathrm{App}(a,b,v,x) &= f_F^{-1}(v)(x). \end{aligned}$$

The formal application is a total set operation; the stated inverse and evaluation laws hold under their proved membership and functionality conditions. `semantic_product_typed`, `semantic_lambda_typed` and `semantic_application_typed` establish the corresponding membership laws. `semantic_beta` gives

$$\mathrm{App}(a,b,\mathrm{Lam}(a,b,g),x) = g(x)$$

for an extensional, correctly typed body family and an argument in the domain. Congruence lemmas cover changes of domain, fiber and body representatives. This is the set-theoretic beta law used later; syntactic beta soundness still requires coherent denotation witnesses.

5 Original denotation and valid valuations

`Denotation.denotes` relates an original scoped term, an original context, a valuation ρ and a set value. It interprets sorts by their codes, variables by lookup, products by the selected product code, and annotated lambdas and applications by the operations above. Its lambda and application clauses retain original typing witnesses for their dependent types. No extra constructor, erased annotation or new typing rule is introduced.

The family premises quantify over every decoded argument. They include extensionality, which cannot be silently inferred from Rocq function identity. `denotes_equal` closes the relation under semantic `EQ`. Renaming and simultaneous substitution are proved for this relation, including binder lifting and original annotations.

Validity is defined by recursion on the original context. The empty context admits every empty valuation. An extension $\Gamma, x : A$ requires a valid parent valuation, a denotation a of A, and $\rho(x) \in \mathrm{El}(a)$. It does not assume soundness of all terms in the context. `valid_valuation_lookup` reconstructs denotation and membership for every original context lookup, including enclosing shifts.

The conclusion `realizes` means that there exist values v, a with denotations of t, T and $v \in \mathrm{El}(a)$. Existence of such witnesses and equality of arbitrary witnesses are separate obligations.

6 The central proof: conversion preserves actual values

6.1 Why ordinary typing induction needs help

The conversion rule changes a term's type along beta equivalence. For semantic membership to survive, the two type denotations must decode to equal sets. Moreover, the type-directed lambda and application clauses must agree when they use different original type witnesses.

`RealizationInduction` first isolates the necessary semantic conversion lemma as an explicit parameter. The final proof does not assume that parameter: `RankedNormalization` constructs it before `ModelSoundness` instantiates the induction. The intermediate conditional theorem should therefore be read together with this later discharge.

6.2 Normal certificates and hereditary substitution

`RankedDenotation.certificate` relates a normal term, its normal type, their actual values and original typing evidence. It entails normality, original typing and the corresponding canonical denotations. Structural coherence is available for these normal objects.

Substitution can expose a new redex even when its inputs are normal. The theorem `hereditary_substitution` in `RankedHereditary` uses an outer well-founded induction on a cut-rank bound, with structural induction on the certificate inside it. There are two application cases. If the substituted head stays neutral, rebuild its application certificate. If it becomes a lambda, recover the body certificate, contract the redex, and normalize the resulting substitution at a strictly smaller rank.

The strict inequality is substantive. `NeutralCutRank` proves that a variable-headed application spine cannot increase the rank of its assigned type. A lambda exposed through substitution consequently has a domain rank strictly below the relevant cut bound, because product formation uses $\max(m+1, n)$. The recursion is thus justified by the original typing rules, not by a termination hypothesis about the desired semantics.

Binder cases also normalize annotations and transport contexts. A family of certificates indexed by decoded arguments must share one syntactic normal form. `uniform_normalization` chooses that form from the original typed term first and uses uniqueness of normal forms to align every family member. The argument still works when the indexing set is empty; it never chooses an element from an empty semantic domain.

6.3 Return to unrestricted original denotation

`original_denotation_normalizes` is proved by induction on arbitrary original denotation evidence. It retains the supplied semantic value while constructing a ranked certificate for a syntactic normal form. The application case invokes certified hereditary application. The binder cases use uniform normalization and context transport.

The required variable certificates are then derived from original validity by induction on context well-formedness. They are not extra user premises. This yields `original_typed_normalization` for every original typed term under an original valid valuation.

Theorem 6.1 (Semantic conversion)**.** *Fix the explicit model parameters described above. If Γ is well formed, ρ is valid, t and u are originally typed, and $t =_\beta u$, then any original denotations v of t and w of u satisfy $v = w$ extensionally.*

Proof. Normalize both denotation witnesses while preserving v and w. Original confluence makes their syntactic normal forms identical. Coherence of canonical denotation on this shared normal form gives $v = w$. The terms may have different displayed types: original typing and conversion, rather than an assumption of identical type witnesses, justify the comparison. □

The public declaration is `original_typed_conversion_values` in `RankedNormalization`. Applying extensionality of decoding gives `original_typed_decoded_conversion`, exactly the lemma required by `RealizationInduction`. Equality of the actual values is stronger than membership in extensionally equal decoded types.

7 Public soundness and nontriviality theorems

Theorem 7.1 (Original typing soundness)**.** *Suppose W is transitive, $\Omega \subseteq W$, H is its supplied tower envelope, and classical excluded middle, cardinal comparability, the infinite powerset gap and family choice hold. For a selector s satisfying* `selects_global_witnesses`*, every original judgment $\Gamma \vdash t : T$ in a well-formed context is realized at every original valid valuation.*

`ModelSoundness.original_typing_sound` is the exact entry point. Its proof instantiates the original realization induction with the conversion theorem just established. The six original typing rules are all covered. `RegularTyping` provides an equivalent enriched derivation convenient for the induction; its conversion from original typing is proved, and the public statement still quantifies over `Typing.typ`.

Three companion declarations explain what the semantics guarantees:

- `original_denotation_coherent` equates arbitrary denotations of the same originally typed term under a valid valuation.
- `original_membership_sound` proves $v \in \mathrm{El}(a)$ for arbitrary denotation witnesses of a typed term and its type, not only the witnesses selected by the existence proof.
- `original_rank_sound` proves that a denotation of a term typed at $\mathcal{U}_k$ belongs to the actual code universe U_k.

The theorem `sound_model_exists` constructs the selector from family choice and packages original realization and membership soundness. It still retains the hierarchy and other set-theoretic parameters.

Theorem 7.2 (Empty Bottom)**.** *Under the same model parameters, the interpretation of the original type* $\mathsf{Bottom} = \Pi Z : \mathcal{U}_0.\, Z$ *is empty. Consequently there is no closed original term t with $\vdash t : \mathsf{Bottom}$.*

Proof. The ordinary dependent product indexed by U_0 has an empty fiber: $c_0 \in U_0$ and $\mathrm{El}(c_0) = \varnothing$. Hence that function set is empty. The chosen bijection transfers emptiness to the decoded product code. A hypothetical typed inhabitant would, by original typing soundness, supply a member of this empty set. Coherence identifies the type witness in that realization with the explicitly constructed denotation of Bottom. □

The declarations are `bottom_interpretation_empty` and `closed_bottom_uninhabited`. The final `sound_nontrivial_model_exists` theorem combines selector existence, original typing soundness and emptiness of Bottom. This is external semantic nontriviality; internal arithmetic refutations are a different result in the arithmetic libraries.

8 Discussion and formalization boundary

The proof turns the informal instruction "choose a code for each product" into a coherent operation independent of typing derivations and rank bounds. Its largest additional obligation is preserving original values through normalization. Normal forms are used to prove agreement of the original semantics, not to replace the original judgment by a new checker or to interpret only a normal fragment.

The construction is completed at its explicit supplied-model interface. A closed transitive assumption report means that the stated implication has no additional global axiom dependencies. It does not remove its arguments for excluded middle, cardinal comparison, the powerset gap, family choice or the supplied sets. In particular it does not prove that these data and deductions exist inside a weaker material set theory.

The separate, ongoing `PTSrkBZCModel` development addresses that question and is outside this documentation's publication scope. The frozen reference model's `BZC_AUDIT.md` and `FOUNDATIONS.md` record the foundation assumptions; their intermediate obligation lists are historical. The intended reduction to material BZC with the stated GCH convention and an actual supplied $V_{\omega+\omega}$ remains open. Consult the live handoff in the full development checkout for subsequent local proof progress. This exposition does not duplicate or declare completion of that excluded development.

Similarly, semantic monotonicity of code universes is not an added cumulativity rule for PTSrk. All theorem statements here use the existing annotated syntax, original beta conversion and original `RankSorts` rules. No arithmetic interpretation, new primitive, eta conversion or assumed semantic soundness is needed to obtain the final soundness theorem.

9 Reproduction and evidence

Use the repository toolchain in `docs/DirectR-CI.md`: OCaml 4.14.2, Dune 3.23.1, Rocq 9.2.0, Stdlib 9.1.0 and Equations 1.3.2+9.2. From the repository root, in that environment:

```
dune build @theories/ZFC_model/audit
```

The alias builds `ModelSoundness` and `MaterialZermelo` and their dependencies. A new absolute Dune build directory is required to rerun the source-level `Print Assumptions` commands. To inspect the public boundary directly:

```
From PTSrkModel Require Import ModelSoundness RankedNormalization.
Check ModelSoundness.original_typing_sound.
Check ModelSoundness.sound_nontrivial_model_exists.
Print Assumptions ModelSoundness.original_typing_sound.
Print Assumptions ModelSoundness.sound_nontrivial_model_exists.
```

The complete staged check is `bash tools/check-local-ci.sh`, which also builds pristine upstream and checks all supported developments. Current commands, exit statuses and assumption evidence are recorded in `docs/direct-r-handoff/STATE.md`; historical logs retain their original paths. They distinguish public theorem dependencies from the larger recursive kernel-check context, which includes unchanged upstream functional extensionality.

Rebuild this exposition and the four companion documents with `bash tools/build-development-docs.sh`. The script uses the same local Markdown–TeX dialect as the manuscript, keeps intermediate files and logs in a fresh temporary directory, and publishes the PDFs only after all five documents compile without unresolved references, missing glyphs or overflowing boxes.